

LECTURE – 8

QUICKSORT

Simple, efficient, in-place, 'stable'.

Simplified basic idea: Suppose the array is partitioned in two parts such that:



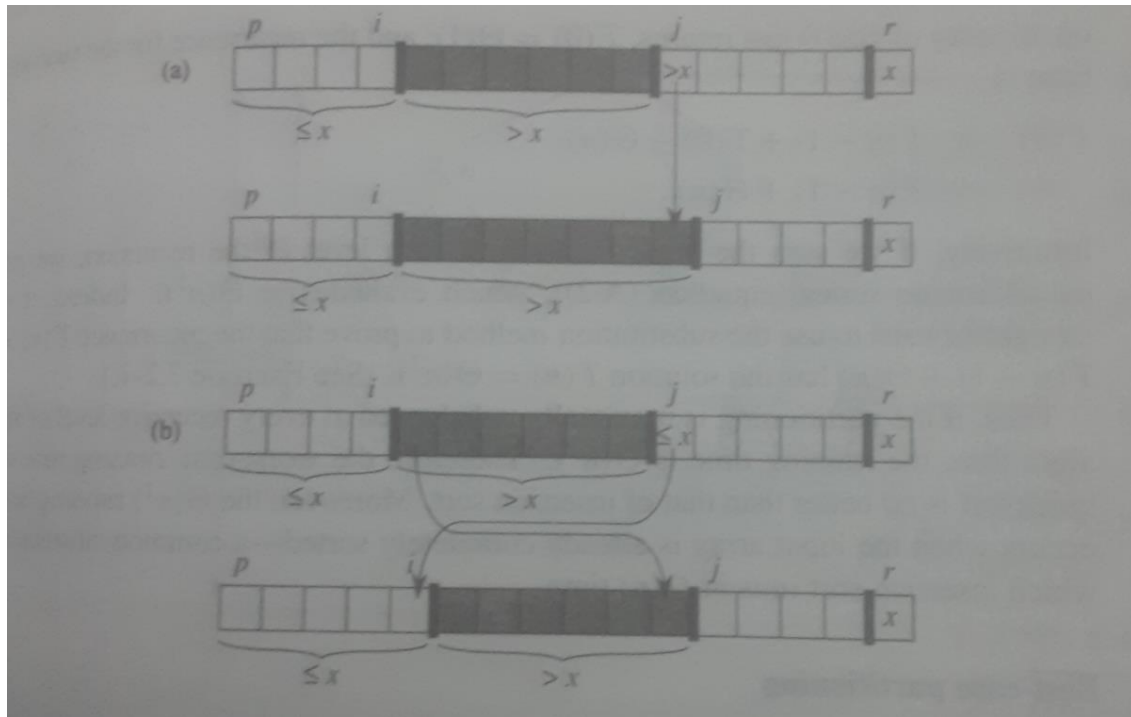
Then the two parts can be sorted independently of each other! **Recursively!**

```
QUICKSORT( A, p, r )
    if p < r                                // significance
        q = PARTITION( A, p, r );
        QUICKSORT( A, p, q-1 );
        QUICKSORT( A, q+1, r );             // index q separates
                                            // the two parts
```

The main question is how to partition the array.

```
PARTITION( A, p, r )
    x = A[r];                                // pivot element
    i = p-1;
    //
    for j = p, r-1
        if A[j] ≤ x
            i = i+1;                          // first part grows: A[j]
            EXCHANGE A[i], A[j];              // goes to first part
    //
    EXCHANGE A[i+1], A[r];                   // move pivot element
                                            // to its right place
    return i+1;                             // and return its index
```

The figure below illustrates the main step in the loop.



Loop invariant:

At the start of each iteration of the **for** loop:

In the range given by $p \leq k \leq i$, $A[k] \leq x$ // smaller values

In the range given by $i+1 \leq k \leq j-1$, $A[k] > x$ // larger values

$A[r] = x$, which is the pivot element.

In the range: $j \leq k \leq r-1$, nothing is known yet about $A[k]$.

Because of the single pass of the **for** loop, the running time of **partition** is easily seen to be $\Theta(n)$, where $n = r - p + 1$. Worst case running time of quicksort is $\Theta(n^2)$. Best case and average case running time of quicksort is $\Theta(n \lg n)$.

Exercise: Array $A[1..8]$ contains the numbers 2, 8, 7, 1, 3, 5, 6, 4. Show that the partition generated as above is 2, 1, 3, 4 | 7, 5, 6, 8 [CLRS Fig. 7.1].

Related points to note:

1. **Randomized** version of **quicksort**.
2. Meaning of **order statistics**.
3. Example of **sort algorithms with $\Theta(n)$ running time**.